



Administración de Redes y Sistemas Computacionales

Laboratorio 5 - Contenedores con Docker y Dockerfile

Prof. Ricardo Pérez
riperez@utalca.cl

Introducción

En este laboratorio asumirás el rol de un desarrollador DevOps encargado de contenerizar una aplicación web sencilla, compuesta por un servidor Nginx que sirve una página HTML y un segundo servicio que monitorea sus logs. El objetivo es construir, configurar y desplegar tus propios contenedores a partir de un **Dockerfile**, sin depender de imágenes preconfiguradas listas para usar.

Restricción: los **Dockerfile** y scripts deben ser escritos por ti, instrucción por instrucción. El informe debe explicar cada línea. El profesor les preguntará sobre cualquier decisión tomada y la respuesta puede afectar la calificación. Un Dockerfile generado íntegramente por IA, sin evidencia de comprensión propia, equivale a la nota mínima.

1. Objetivos

1. Comprender la arquitectura de Docker: imágenes, contenedores, capas y registros.
2. Crear imágenes personalizadas mediante **Dockerfile**.
3. Gestionar variables de entorno, volúmenes y mapeo de puertos.
4. Aplicar buenas prácticas de construcción de imágenes.
5. Diagnosticar y resolver fallos reales de build y de despliegue.

2. Contexto del proyecto

La empresa ficticia *SoftDev Ltda.* necesita desplegar su sitio web corporativo en contenedores. Debes contenerizar dos servicios:

1. **web:** servidor Nginx que sirve una página HTML personalizada (nombre del estudiante, empresa ficticia, fecha de despliegue vía variable de entorno).
2. **logger:** contenedor Alpine que monitorea el log de acceso del servidor web mediante un script de shell y escribe un resumen en un volumen compartido.

Listing 1: Estructura de directorios del proyecto

```
proyecto-docker/  
  web/  
    Dockerfile  
    nginx.conf  
    html/  
      index.html  
  logger/  
    Dockerfile  
    monitor.sh  
  volumes/  
    logs/  
  README.md
```

3. Referencia rápida

Instrucción	Uso
FROM	imagen base
RUN	ejecuta comandos durante el build
COPY	copia archivos al contexto de la imagen
WORKDIR	directorio de trabajo
ENV	variables de entorno (build y runtime)
ARG	variables solo de build
EXPOSE	documenta el puerto usado
CMD / ENTRYPOINT	comando por defecto al iniciar
VOLUME	punto de montaje persistente
LABEL	metadatos de la imagen

Comandos que usarás con frecuencia: `docker build`, `docker run`, `docker ps -a`, `docker logs`, `docker exec -it`, `docker inspect`, `docker history`, `docker volume`, `docker network`. Documentación completa en <https://docs.docker.com/reference/dockerfile/>.

4. Parte 1 - Preparación (10 pts)

1.1. Verifica tu instalación y captura la salida:

```
docker --version
docker info
docker run hello-world
```

Responde: ¿qué proceso siguió Docker al ejecutar `hello-world` (búsqueda local, descarga, ejecución)?

1.2. Crea la estructura de directorios de la Sección 2 y verifícala con `tree` o `find`.

5. Parte 2 - Servicio web (30 pts)

2.1. Crea `web/html/index.html` con tu nombre, la empresa ficticia y un mensaje de estado. CSS embebido, sin mayor complejidad visual.

2.2. Crea `web/nginx.conf`: escucha en el puerto 80, sirve archivos estáticos y define una página 404 propia.

2.3. Escribe `web/Dockerfile` que:

- use `nginx:alpine` como base;
- agregue LABEL (autor, versión 1.0, descripción);
- defina `ENV DEPLOY_DATE` y `ENV EMPRESA`;
- copie `nginx.conf` y el directorio `html/` a su ruta correcta;
- exponga el puerto 80;
- arranque Nginx en modo *foreground* (investiga por qué esto es necesario en Docker).

Explica en el informe qué hace cada línea de tu Dockerfile.

2.4. Construye y ejecuta:

```
docker build -t softdev-web:1.0 ./web
docker run -d --name web-service -p 8080:80 \
  -e DEPLOY_DATE="2026-10-01" \
  -v ./volumes/logs:/var/log/nginx \
  softdev-web:1.0
```

Verifica en <http://localhost:8080>. Entrega: Dockerfile, nginx.conf, index.html, salida de `docker build`, captura del navegador, `docker ps`.

6. Parte 3 - Servicio logger (25 pts)

3.1. Escribe `logger/monitor.sh`: en un bucle infinito, cada 10 segundos lee `/logs/access.log`, cuenta las líneas y agrega (sin sobrescribir) una línea a `/logs/resumen.txt` con el formato `[FECHA HORA] Total de peticiones: N`. Debe imprimir lo mismo por salida estándar.

3.2. Escribe `logger/Dockerfile` que:

- use `alpine:3.18` como base;
- instale `bash`;
- defina `ENV LOG_INTERVAL=10`;
- cree `/logs` y lo declare como `VOLUME`;

- copie `monitor.sh` y le dé permisos de ejecución;
- use `ENTRYPOINT` para ejecutarlo siempre.

Responde: ¿qué diferencia hay entre `CMD` y `ENTRYPOINT`? Da un ejemplo propio para este caso particular.

3.3. Construye y ejecuta montando el mismo volumen de logs que usa `web-service`. Genera tráfico con `curl` y verifica que `resumen.txt` se actualiza. Entrega: `Dockerfile`, `monitor.sh`, salida de `build`, `docker logs logger-service`, contenido de `resumen.txt`, `docker ps` con ambos contenedores activos.

7. Parte 4 - Inspección y ciclo de vida (20 pts)

4.1. Ejecuta y documenta:

```
docker history softdev-web:1.0
docker inspect web-service --format='{{json .Config.Env}}'
docker inspect web-service --format='{{json .Mounts}}'
docker inspect web-service --format='{{.NetworkSettings.IPAddress}}'
```

Responde: ¿cuántas capas tiene tu imagen y a qué instrucción corresponde cada una? ¿Qué IP interna tiene el contenedor?

4.2. Entra de forma interactiva (`docker exec -it web-service sh`) y revisa variables de entorno, el archivo de configuración copiado y el resultado de `nginx -t`.

4.3. Practica el ciclo completo: `stop`, `ps -a`, `start`, verificación con `curl -I`, y finalmente `rm` de ambos contenedores. Responde: ¿`stop` elimina la imagen o los datos del bind mount? ¿Qué hace `docker system prune`?

8. Parte 5 - Diagnóstico y gestión avanzada (15 pts)

A partir de aquí, cada ejercicio describe un problema o una restricción sobre tu propio entorno. No hay un comando único que lo resuelva de memoria: necesitas observar el estado real de tus contenedores, imágenes y volúmenes.

5.1 - Build roto por orden de capas. Modifica tu `web/Dockerfile` para que `COPY html/ /usr/share/nginx/html` ocurra *antes* de una instrucción `RUN` que falle a propósito (por ejemplo, un paquete inexistente con `apk add paquete-que-no-existe`). Ejecuta `docker build` y observa en qué capa se detiene. Corrige el `Dockerfile` y reconstrúyelo. Con `docker history`, compara el número y el peso de las capas antes y después del arreglo. Explica, con tus propios resultados (no en general), por qué el orden de las instrucciones afecta el caché de build.

5.2 - Conexión rota entre web y logger. Rompe intencionalmente la comunicación entre tus dos servicios: monta `logger` apuntando a un volumen distinto al que usa `web-service`, o detén `web-service` antes de iniciar `logger`. Usa `docker logs` e `inspect` sobre ambos contenedores para identificar la causa exacta (ruta de volumen, orden de arranque, permisos). Corrige el problema sin perder el contenido ya escrito en `resumen.txt`. Entrega capturas del error, el diagnóstico paso a paso y la corrección funcionando.

5.3 - Límite de recursos y OOM. Ejecuta un contenedor nuevo con `-memory` limitado (por ejemplo 128m) y, dentro de él, genera carga de memoria por encima del límite (`stress-ng` o un script simple que reserve memoria). Observa `docker stats` mientras ocurre, y revisa después `docker inspect` buscando el campo `OOMKilled` y el `exit code`. Explica qué pasó en tu caso concreto.

5.4 - Backup y restauración de un volumen. Respaldar el volumen de logs en un `.tar.gz` usando un contenedor temporal (`alpine` o `busybox`) que monte el volumen de origen y un directorio destino en el host. Elimina el volumen original, créalo de nuevo con el mismo nombre y restaura el respaldo con el mismo mecanismo. Confirma con el contenido real de `resumen.txt` que los datos sobrevivieron al ciclo completo.

5.5 - Reducir una imagen con multi-stage build. Toma una imagen que hayas construido con herramientas de compilación (por ejemplo, si instalaste un compilador o SDK en algún ejercicio o laboratorio anterior, o construye un binario simple en C o java para este ejercicio). Reescríbela como un `Dockerfile` de dos etapas: una etapa de build con la imagen pesada, y una etapa final mínima que solo copie el artefacto resultante con `COPY --from=`. Compara el tamaño final con `docker images` y documenta la diferencia real en MB.

9. Informe

Entrega en PDF: portada, introducción breve, desarrollo por parte (código fuente completo, explicación línea por línea de cada `Dockerfile`, respuestas a las preguntas, evidencias con descripción), un problema real que hayas enfrentado y cómo lo resolviste, conclusiones, y referencias.

Referencias

- Documentación oficial de Docker: <https://docs.docker.com>
- Referencia de `Dockerfile`: <https://docs.docker.com/reference/dockerfile/>
- Docker Hub: <https://hub.docker.com>
- Buenas prácticas de build: <https://docs.docker.com/build/building/best-practices/>